

Ransomware Sentinel

Technical Project Report

Local behavioural ransomware detection and incident response for Windows

Version 1.3.0

Python 3.13 · PySide6 · watchdog · psutil · SQLite

Contents

Contents.....	2
Abstract	3
1. Introduction	3
2. Problem statement.....	3
3. Objectives	3
4. Threat model	4
5. System requirements.....	4
6. System architecture.....	4
7. Module architecture	5
8. Automatic protection	5
9. Initial scanning	6
10. Filesystem monitoring	6
11. Process monitoring.....	7
12. Behavioural detection.....	7
13. Risk scoring.....	7
14. Evidence collection	7
15. Alert and incident lifecycle	7
16. Response	8
17. Quarantine	8
18. Audit integrity.....	9
19. Recovery readiness	9
20. Security posture and guidance	9
21. GUI architecture	10
22. Safe demonstration	10
23. Testing strategy.....	11
24. Test results	11
25. Security considerations	12
26. Limitations.....	12
27. Future work.....	12
28. Conclusion.....	12

(If the contents list is empty, right-click it in Word and choose Update Field.)

Abstract

Ransomware Sentinel is a local, user-mode Windows desktop application that detects the file-system behaviour typical of ransomware in a user's own folders, explains its assessment, and supports investigation and response. File-system events from protected locations are evaluated by five deterministic rules over a 30-second sliding window; their contributions form an explainable 0–100 risk score that is mapped to a severity band by a single risk engine. Alerts are correlated into incidents with a validated lifecycle, response actions (acknowledge, investigate, quarantine, restore, trust, resolve, dismiss) are user-initiated and recorded with their real outcome, and security-relevant records are committed to an HMAC-SHA256 chained audit log. Version 1.3 adds automatic protection of the user's data folders, an initial scan before monitoring, a guided first-run experience and a safe demonstration mode. The system is validated by an automated suite of 1391 tests (1389 passed, 2 skipped, 0 failed). It provides visibility and response, not kernel-level protection, and it does not prevent, decrypt or recover encrypted files.

1. Introduction

Ransomware encrypts or replaces a victim's files and demands payment for their return. Its defining observable behaviour on a workstation is a burst of file modifications and renames, often to unfamiliar extensions, within a short time. Ransomware Sentinel is built around that observation: it watches behaviour in the folders that matter to the user, explains what it saw, and gives the user the tools and records to respond.

The project was developed as a Windows-first, local-first application with an emphasis on correctness, explainability and honest reporting of coverage. This report describes the implementation as it stands, its testing, and its limitations.

2. Problem statement

- Signature-based detection recognises known samples; behaviour can be observed for unknown variants.
- A bare verdict does not tell a user what happened, which files were affected, or what to do next.
- Security tools frequently overstate their coverage; a user cannot tell which folders are actually watched.
- Response actions taken under pressure need a durable, verifiable record.
- A defender needs a safe way to demonstrate and practise the workflow without real malware.

3. Objectives

1. Detect ransomware-like behaviour in user-chosen folders using explainable, deterministic rules.
2. Express the assessment as a 0–100 score whose every point is attributable to a named rule.
3. Correlate alerts into one incident per episode, with evidence and a validated lifecycle.
4. Provide confirmed, audited response actions, including containment of an attributed executable.
5. Record security-relevant state in a tamper-evident audit log verified at start-up.
6. Report the real state of protection honestly, including degraded and unknown states.
7. Protect the user's data folders automatically and guide a new user from setup to response.

8. Offer a fully synthetic, safe demonstration of the complete workflow.

4. Threat model

In scope: a process running with the user's privileges that rapidly modifies, renames or creates files inside protected locations — the behaviour of ransomware operating on user data.

Out of scope, by design: software running as the same user that deliberately targets Sentinel (it can terminate the process or forge its records using the same DPAPI-protected key), administrator, SYSTEM or kernel-mode attackers, activity outside protected locations, network-share encryption from other hosts, and data exfiltration. The audit chain detects modification of the records it covers after the fact; it cannot prevent it.

5. System requirements

Area	Requirement
Platform	Windows 10/11 (developed and tested on Windows 11), user privileges only
Runtime	Python 3.13; PySide6, watchdog, psutil (requirements.txt)
Storage	SQLite (standard library), WAL mode; data folder per user
Tests	pytest, pytest-randomly, pytest-timeout (requirements-dev.txt)
Network	None at run time; no account, cloud service or telemetry upload

6. System architecture

Sentinel is one desktop process. `main.py` is the composition root: it loads settings, acquires the single-instance lock, and constructs the `ApplicationController` and `MainWindow`. The controller orchestrates the application lifecycle and is the single join point between domain coordinators and the GUI, which communicate through Qt signals.

Layer	Responsibility
GUI (src/gui)	Renders state it is given and emits intent signals. Makes no security decisions; contains no SQL.
ApplicationController	Lifecycle; joins coordinators; publishes state; carries out user intents.
Coordinators (src/coordinators)	Monitoring, Protection, ProtectionFlow, Detection, Incidents, SecurityState, Demo — peers that never import one another.
Engines and stores	Behaviour engine, risk engine, alert manager, incident manager, quarantine, trust, integrity chain, SQLite.
Sensors	watchdog file monitor and psutil process monitor (user mode).

These boundaries are enforced by architecture tests that parse the source: a GUI module importing sqlite3, the database manager, a coordinator, a detection engine or the posture evaluator fails the build, as does a coordinator importing another coordinator.

7. Module architecture

Module	Contents
src/monitors	file_monitor (watchdog), process_monitor (psutil), event_intake (admission control), on_demand_scanner
src/protection	scope (protected-location model), scope_health (asynchronous probes), locations (default folder discovery)
src/detection	behavior_engine, rules, risk_engine, attribution
src/alerts, src/incidents	alert_manager; incident_model, incident_manager, file_activity
src/response, src/trust	quarantine; trust_model
src/security	integrity (HMAC chain and anchor), keyring (DPAPI), config_integrity
src/recovery	readiness assessment of backup locations
src/database	database_manager, telemetry_store, retention
src/simulator	safe_demo simulator, worker, demo_workspace
src/controllers	application_controller, security_posture, guidance
src/models	value types published to the GUI

8. Automatic protection

Version 1.3 proposes the user's own data folders as protected locations: Desktop, Documents, Downloads, Pictures, Videos and Music. They are resolved with the Windows Known Folder API (SHGetKnownFolderPath), so a folder redirected by OneDrive Known Folder Move or to another drive is found at its real location; when the API has no answer, %USERPROFILE%\<Name> is used. OneDrive is detected from the OneDrive environment variables and proposed switched off, because its sync client produces bursts of creations and renames. Drive roots and system folders are never proposed; adding one manually shows a warning.

Each location is a ProtectedScope with a normalised identity, an enabled flag and a label. Its health is probed off the GUI thread and reported as OK, DISABLED, MISSING, NOT_A_DIRECTORY, PERMISSION_DENIED, REDIRECTED (junction, not followed), OVERLAPPING, INVALID, CHECKING or UNREACHABLE. A location is never silently removed because it is unavailable. When one healthy location contains another, the containing one is watched and the contained one is reported as covered.

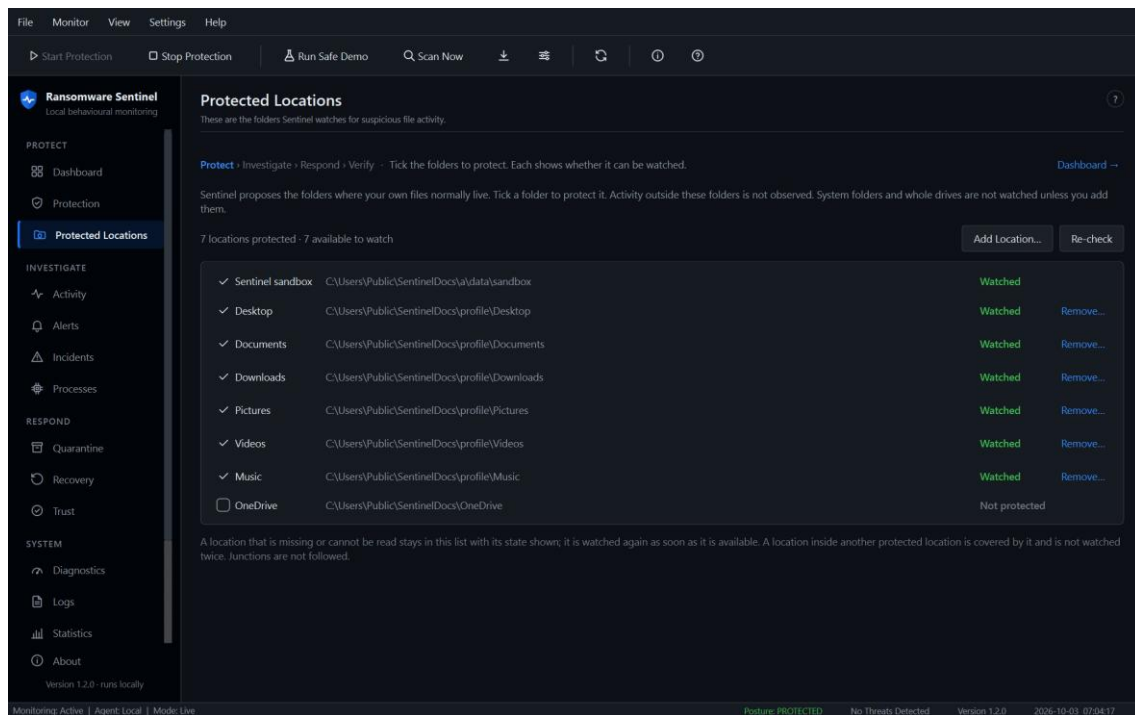


Figure 1. Protected Locations

Start Protection is coordinated by the ProtectionFlowCoordinator through the phases MONITORING OFF, INITIALIZING (location health awaited, with a 15-second deadline so an unreachable network folder cannot hold protection back), INITIAL SCAN, MONITORING ACTIVE, STOPPING and ERROR.

9. Initial scanning

The initial scan reuses the on-demand scanner infrastructure. A shared walker enumerates files with `os.scandir`, does not follow symbolic links or directory junctions (`os.walk` follows junctions on Windows even with `followlinks=False`), and counts unreadable folders as skipped. The `ProtectedLocationScanWorker` streams across all watchable roots on a worker thread, reports running totals (files examined, suspicious names, skipped), and can be cancelled. It checks file names against the configured suspicious extensions and never opens or changes file contents.

The scan runs before continuous monitoring starts, so enumerating existing files can never be observed or scored as behaviour. This is verified by an end-to-end test that asserts no file events reach the detector during the scan.

10. Filesystem monitoring

A single watchdog observer schedules one recursive watch per root. Roots are reduced to the outermost set, so a folder inside another watched folder is not watched twice and events are not double-counted. Sentinel's own files — the database and its WAL/SHM files, settings, logs, the audit state and the quarantine store — are excluded from monitoring, so its own writes cannot feed back into detection when the data folder lies inside a protected location; the Demo Center workspace remains observable. Raw events are handed to the GUI thread through a queued signal and pass through event intake, which under overload sheds only repeat modifications of already-seen paths (never renames, creations or deletions), counts what it sheds, and degrades the posture.

11. Process monitoring

A psutil-based process monitor takes periodic snapshots on a worker thread for the Processes page and for attribution context. It is system context, not a verdict: no risk is computed per process.

12. Behavioural detection

Rule	Default trigger (30 s window)	Points
RAPID_FILE_MODIFICATION	50 modifications	25
MASS_FILE_RENAME	20 renames	25
SUSPICIOUS_EXTENSION	at least one rename to a suspicious extension	20
MASS_FILE_CREATION	50 creations	15
HIGH_EVENT_CONCENTRATION	75 events	15

The rules are deterministic functions of the sliding window; no machine-learning model is used. Thresholds are configurable in Settings and pinned by tests so they cannot change unnoticed.

13. Risk scoring

The risk engine sums rule contributions into a 0–100 score and is the only component that maps a score to a severity band: NONE (0), LOW (1–29), MEDIUM (30–59), HIGH (60–79) and CRITICAL (80–100) by default. The GUI never re-derives a band. An alert is raised at HIGH or above, with a cooldown that never holds back an escalation to CRITICAL.

14. Evidence collection

Every triggered rule contributes an evidence item (rule, points, observation, event count), and the evidence shown always sums to the score shown. Per-file activity is reconstructed from recorded events, following renames as one file. Attribution runs when an alert fires and states its confidence explicitly — UNKNOWN, UNAVAILABLE or UNCONFIRMED; process identity is (pid, create_time), so a reused PID is not mistaken for the original process.

15. Alert and incident lifecycle

Alerts from one episode are correlated into one incident (same process identity, or the most recent unattributed active incident within five minutes). An incident records the peak severity of its episode and follows a validated status machine: OPEN → INVESTIGATING → CONTAINED/QUARANTINED → RESOLVED, or DISMISSED. Each incident's forensic fields are committed to the audit chain, and the investigation view reports VERIFIED, ALTERED, UNCOMMITTED or UNVERIFIABLE.

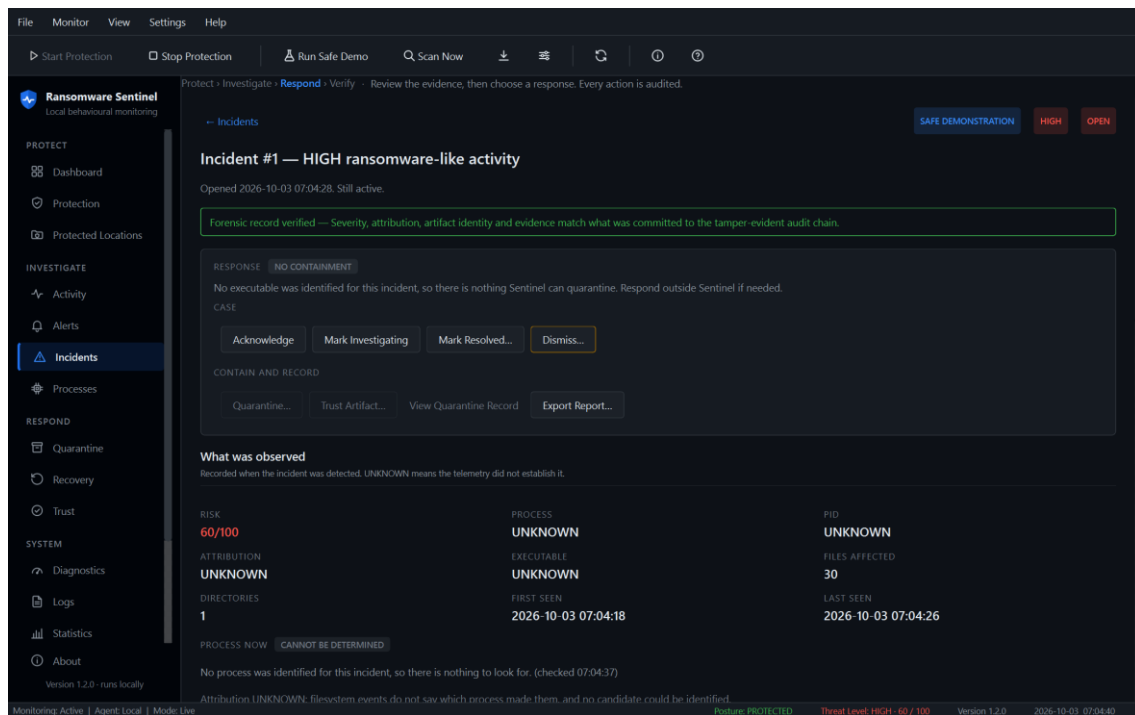


Figure 2. Incident detail

16. Response

All response actions are user-initiated, confirmed in a dialog that names the target and defaults to Cancel, and recorded in the incident's audit trail with their real outcome (SUCCEEDED, FAILED, REFUSED, TARGET_MISSING, PERMISSION_DENIED). Actions: acknowledge, mark investigating, resolve, dismiss, export a JSON report, quarantine and restore, and trust and revoke trust. Trust is bound to the SHA-256 of an executable, with a mandatory reason, optional expiry and revocation; it stops trusted activity driving the posture but does not stop detection.

17. Quarantine

Quarantine moves the attributed executable into Sentinel's store under an opaque name with a .quarantine suffix, hashing the file before the move and the stored copy afterwards. Metadata is written write-then-rename and committed to the audit chain. System locations and Windows device names are refused. Restore verifies the stored bytes against the recorded hash and refuses to overwrite an existing file. Quarantine is containment, not isolation: a process running as the same user can still read the stored bytes.

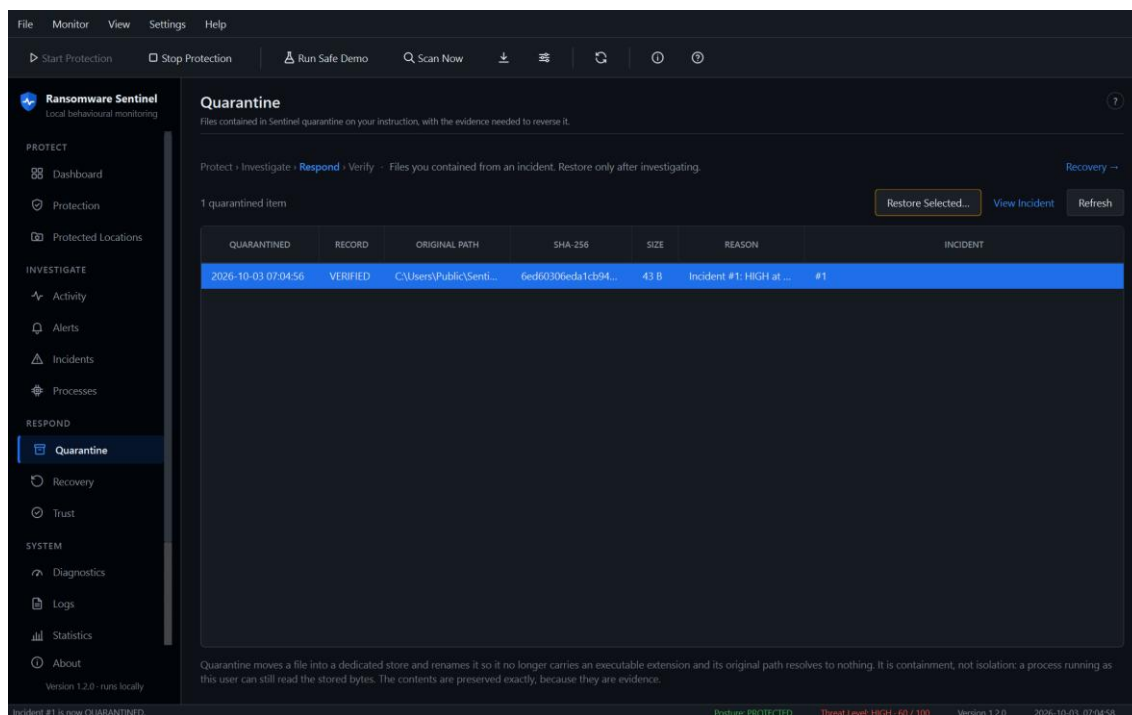


Figure 3. Quarantine (documentation data: a harmless file stands in for an attributed executable)

18. Audit integrity

Security-relevant acts (trust, incident, quarantine, retention and configuration records) are appended to a chain in which each entry's MAC is HMAC-SHA256(key, entry || previous MAC). An anchor file outside the database records the latest sequence number and MAC, so truncation and rollback are detected. The key is protected with Windows DPAPI. The chain is verified at start-up and on demand (Diagnostics → Verify Now); security-relevant configuration is verified as well.

19. Recovery readiness

Sentinel takes no backups and cannot decrypt files. The recovery readiness assessor inspects the backup locations the user configures, read-only and bounded, and reports RECOVERY READY, RECOVERY PARTIAL, NO VERIFIED RECOVERY SOURCE or RECOVERY UNKNOWN. It is an observation that an independent copy appears to exist, not a recovery.

20. Security posture and guidance

A pure function, `evaluate_posture`, turns observed facts (monitor liveness, engine readiness, active critical incidents, database availability, telemetry shedding, scope coverage, integrity state) into one posture: CRITICAL THREAT, ERROR, DEGRADED, MONITORING, MONITORING OFF or PROTECTED. PROTECTED means the monitoring pipeline is fully working on the protected locations, not that the machine is safe, and its wording names any incident still open. A second pure function, `next_action`, turns the same facts into one recommended step shown on the Dashboard, most urgent first: open an incident, fix an error, run setup, start protection, fix a location, address degradation, add a backup location, or try the demo.

21. GUI architecture

The interface is a PySide6 main window with grouped navigation — Protect, Investigate, Respond, System and Learn — over seventeen pages and the incident view. Pages render value objects published by the controller and emit intent signals. Version 1.3 adds a first-run setup wizard, a guided tour that highlights real controls (scrolling them into view and never outlining a hidden component), a one-line workflow hint on each page, a Getting Started hub, contextual help buttons, and a protection panel and next-step banner on the Dashboard. Untrusted text (file names, paths, process names) is neutralised at the presentation boundary only.

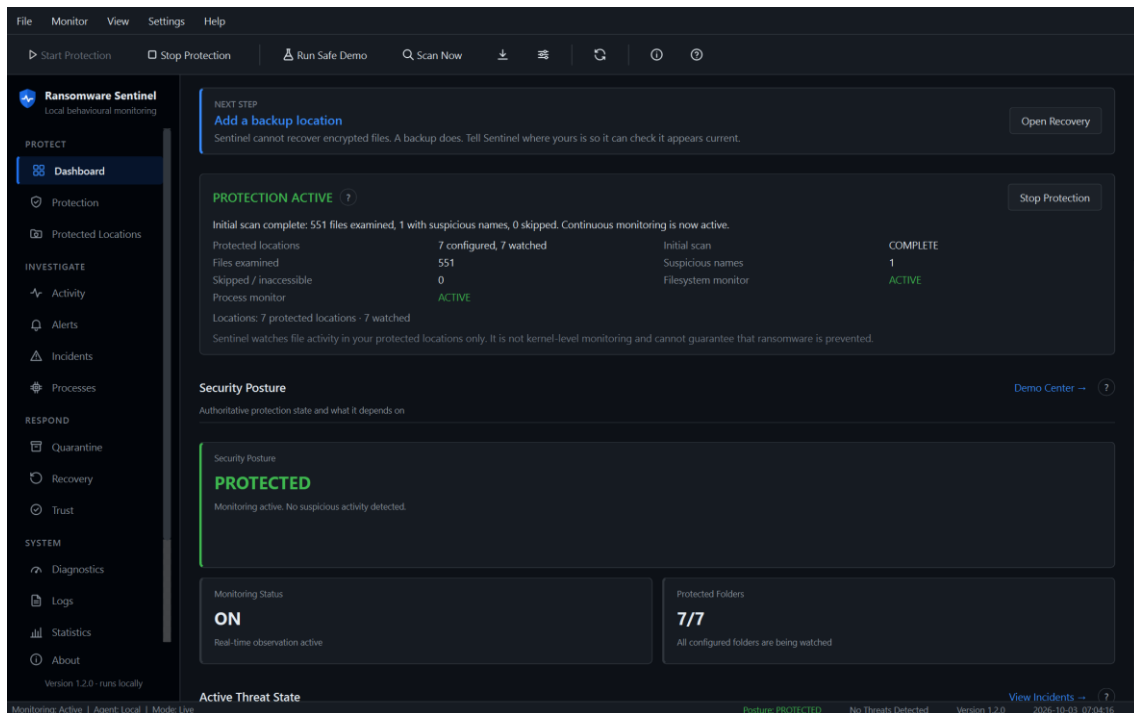


Figure 4. Dashboard

22. Safe demonstration

The Demo Center runs the existing safe simulator against an isolated workspace inside Sentinel's data folder. The simulator creates, modifies and renames inert text files and records every file and rename in a manifest kept outside the watched folder. The real pipeline does everything else. An incident is annotated as a demonstration only if every file in its recorded activity lies inside the workspace; the demo refuses to start while another incident is active, so its alert cannot be merged into a real case. Reset removes only manifest-listed files, one by one and never recursively, and dismisses the demo incident through the normal audited transition; records are kept. Simulation and detection have deadlines and fail with what was observed.

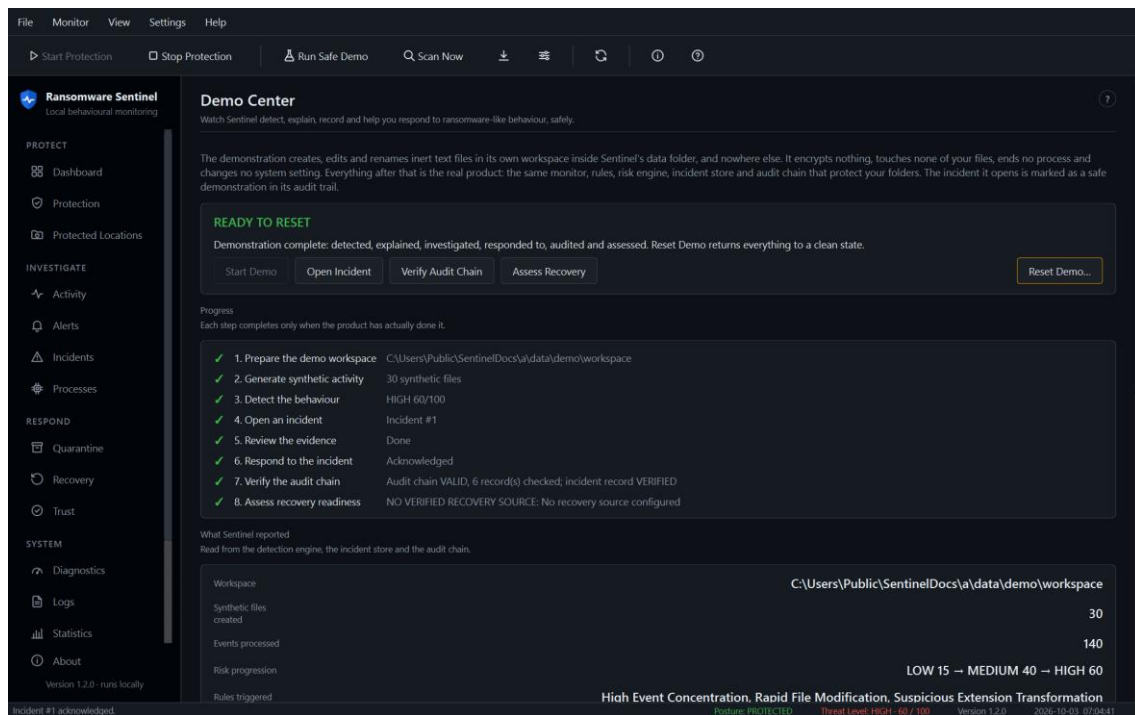


Figure 5. Demo Center after a complete demonstration

23. Testing strategy

- **Unit tests** of rules, risk engine, attribution, trust, quarantine, recovery, retention, settings and the scope model.
- **Pinned detection semantics** so thresholds and points cannot change without a test changing.
- **A benign corpus** of ordinary activity (editor saves, builds, archive extraction, photo import) that must not reach CRITICAL.
- **Integration tests** that drive the real controller, database and file monitor end to end.
- **Architecture tests** enforcing the GUI and coordinator boundaries.
- **Hostile-input and failure-injection tests** covering control characters, database failures and unreadable stores.
- **Protection and productization tests** covering discovery, the initial scan, monitoring, setup, the guided tour at three window sizes, guidance and the demo walkthrough.

The suite runs in random order (pytest-randomly) with a 150-second per-test timeout (pytest-timeout), and every test gets its own temporary data folder.

24. Test results

Measure	Result
Tests collected	1391
Result	1389 passed, 2 skipped, 0 failed
Random seed	Not recorded (quiet run)
Duration	749.3 s (randomised order, 150 s per-test timeout)
Environment	Windows 11, Python 3.13.7, PySide6 6.11.2, pytest 9.1.1

The two skipped tests create symbolic links, which Windows permits only with administrator rights or Developer Mode. In addition to the automated suite, the launched application was exercised by scripted GUI checks: first-run setup end to end, the guided tour, a returning user's automatic start, the Demo Center from start to reset, second-instance refusal and clean shutdown with no orphan processes.

25. Security considerations

- Paths in configuration are normalised and validated; relative paths and oversized values are refused.
- Reparse points are not followed by default when watching or scanning.
- All SQL is parameterised; identifiers come only from fixed allow-lists.
- No shell=True, eval, exec, pickle or subprocess in the application; no network client.
- Quarantine refuses system locations and Windows device names.
- The demo can only delete files it recorded creating inside its own workspace.
- Diagnostics exports contain no key material or file contents.
- Every response is confirmed and audited with its real outcome; success is never assumed.

26. Limitations

- User mode only: no kernel driver, ETW consumer or minifilter.
- Only protected locations are observed.
- Process attribution is usually UNKNOWN; UNCONFIRMED candidates are not proven cause.
- No process termination, suspension or write blocking; no guarantee of prevention.
- No decryption or recovery of encrypted files.
- Quarantine is containment, not isolation.
- Tampering is detected after the fact, not prevented; raw telemetry is not individually authenticated.
- Monitoring is not running during the initial scan; the scan has no time limit (it is cancellable).
- Sync clients such as OneDrive can produce activity resembling the rules.

27. Future work

- Kernel or ETW telemetry for causal process attribution.
- Process suspension as a confirmed response action.
- Volume Shadow Copy inspection to strengthen recovery readiness.
- A signed installer (MSI/MSIX).
- Authenticated telemetry checkpoints.
- Including the start-on-launch preference in the configuration integrity digest with a migration.

28. Conclusion

Ransomware Sentinel shows that a user-mode application can detect ransomware-like behaviour in the folders that matter, explain every point of its assessment, support a complete investigation and

response workflow with a tamper-evident record, and guide a new user from setup to response — while stating plainly what it does not cover. The safe Demo Center allows the full workflow to be demonstrated without real malware and leaves the system clean.